

Advanced Parallel Grid Processing with MPI

Short Documentation Report

Parallel Computing — Practical Project (V1.0, 2026)
Faculty of Computers and Artificial Intelligence — Fayoum University

Language / Library	C++14 + MPI (MPICH / OpenMPI)
Category A algorithm	Heat Diffusion (2D 4-point Stencil)
Category B algorithm	Prefix Sum (Inclusive Scan)
Advanced comm pattern	Collective-based (Bcast / Scatterv / Allgather / Exscan / Gatherv)
Blocking comm	MPI_Send / MPI_Recv
Non-blocking comm	MPI_Isend / MPI_Irecv + MPI_Waitall
Process grouping	MPI_Comm_split
Bonus features	None (per request)
Build / run	make && mpiexec -n P ./pc_project ...

1. Design Overview

The project is delivered as a single MPI executable, **pc_project**, which lets the user choose the algorithm to run at **runtime** through a command-line argument. This satisfies the “multiple parallel algorithms with runtime selection” requirement of the project sheet.

1.1 Project Structure

```
PC2026_Project/
  ■■ Makefile
  ■■ README.md
  ■■ generate_data.py    # creates big input files
  ■■ run_demo.sh        # end-to-end demo
  ■■ src/
    ■ ■■ main.cpp        # entry point, runtime selection
    ■ ■■ heat_diffusion.cpp # Category A
    ■ ■■ prefix_sum.cpp   # Category B
    ■ ■■ deadlock_demo.cpp # deadlock + fix
  ■■ data/              # input / output files
  ■■ report/            # this PDF + screenshots
```

1.2 Algorithm 1 — Heat Diffusion (Category A)

A classical 2-D heat equation is solved using an iterative **4-point stencil**:

$$T_{\text{new}}[i, j] = (T[i-1, j] + T[i+1, j] + T[i, j-1] + T[i, j+1]) / 4$$

- The $R \times C$ grid is decomposed by **rows** across all MPI processes.
- Each process holds *localRows* data rows plus **2 ghost rows** for halo exchange with its top and bottom neighbours.
- **Dirichlet boundary conditions**: the outermost rows and columns of the global grid are held fixed throughout the iterations.
- **Uneven distribution** is handled with `MPI_Scatterv` / `MPI_Gatherv` whenever R is not divisible by P .

1.3 Algorithm 2 — Prefix Sum (Category B)

An inclusive parallel scan is implemented in three steps:

1. **Local scan** — each process computes the prefix sum of its own chunk.
2. **MPI_Exscan** (collective) — every process obtains the sum of *all preceding* ranks.
3. **Offset addition** — the received offset is added to every local prefix value, yielding the global inclusive scan.

Data distribution uses **MPI_Scatterv**, and the final result is reassembled on rank 0 via **MPI_Gatherv**. Uneven splits are fully supported.

1.4 Process Organisation (MPI_Comm_split)

Both algorithms call **MPI_Comm_split** to organise processes into logical groups (even/odd ranks for Heat Diffusion, lower/upper half for Prefix Sum). The sub-communicators are used for logical grouping; data communication still happens on `MPI_COMM_WORLD` to keep the code simple, as requested.

1.5 Big-Data Input

Inputs are generated by **generate_data.py**:

- `data/heat_input.txt` — a 200×200 grid (40000 cells)

- data/prefix_input.txt — an array of 1000000 integers

2. Communication Strategies

The project implements **three** MPI communication strategies. The Heat Diffusion algorithm can run any of them at runtime by passing the chosen mode as the last CLI argument:

2.1 Blocking Communication

Halo rows are exchanged with paired **MPI_Send / MPI_Recv** calls. To prevent the well-known “send-send deadlock”, even-rank and odd-rank processes execute the calls in **reversed order**:

```
if (rank % 2 == 0) {
    MPI_Send(top, ..., up); MPI_Recv(top_ghost, ..., up);
    MPI_Send(bot, ..., down); MPI_Recv(bot_ghost, ..., down);
} else {
    MPI_Recv(bot_ghost, ..., down); MPI_Send(bot, ..., down);
    MPI_Recv(top_ghost, ..., up); MPI_Send(top, ..., up);
}
```

2.2 Non-blocking Communication

All four halo transfers are posted with **MPI_Isend / MPI_Irecv** and a single **MPI_Waitall** at the end. This allows the MPI library to overlap sends/receives freely and removes any ordering constraint:

```
MPI_Request reqs[4];
MPI_Isend(top, ..., up, &reqs[0]);
MPI_Isend(bot, ..., down, &reqs[1]);
MPI_Irecv(top_ghost, ..., up, &reqs[2]);
MPI_Irecv(bot_ghost, ..., down, &reqs[3]);
MPI_Waitall(4, reqs, MPI_STATUSES_IGNORE);
```

2.3 Collective-Based Communication (advanced pattern)

This is the **advanced pattern** required by the project sheet. For Heat Diffusion, every process packs its top and bottom rows into a single buffer and calls **MPI_Allgather**; afterwards each process picks its two neighbouring ghost rows directly from the gathered buffer:

```
std::memcpy(&myEdges[0], &local[1*C], C*sizeof(double));
std::memcpy(&myEdges[C], &local[localRows*C], C*sizeof(double));
MPI_Allgather(myEdges, 2*C, MPI_DOUBLE,
              allEdges, 2*C, MPI_DOUBLE, MPI_COMM_WORLD);
```

For Prefix Sum, the **entire** algorithm is collective: **MPI_Bcast** (size), **MPI_Scatterv** (data), **MPI_Exscan** (running offset) and **MPI_Gatherv** (final assembly). No point-to-point call is used.

2.4 Summary Table

Strategy	Where used	MPI calls
Blocking	Heat Diffusion (mode = blocking)	MPI_Send / MPI_Recv
Non-blocking	Heat Diffusion (mode = nonblocking)	MPI_Isend / MPI_Irecv / MPI_Waitall
Collective	Heat Diffusion (mode = collective) + entire Prefix Sum	MPI_Bcast, MPI_Scatterv, MPI_Allgather, MPI_Exscan, MPI_Gatherv
Process group	Both algorithms	MPI_Comm_split
Deadlock fix	deadlock_demo.cpp	MPI_Sendrecv

3. Deadlock Explanation

3.1 The Broken Scenario

We intentionally construct a classic deadlock between two processes. Each rank calls **MPI_Send** first and then **MPI_Recv** with a large message:

```
// BROKEN VERSION (kept commented out in the source)
if (rank == 0 || rank == 1) {
    int partner = 1 - rank;
    MPI_Send(sendBuf, N, MPI_INT, partner, 0, MPI_COMM_WORLD); // (1)
    MPI_Recv(recvBuf, N, MPI_INT, partner, 0, MPI_COMM_WORLD, // (2)
             MPI_STATUS_IGNORE);
}
```

3.2 Why It Happens

MPI_Send is a **blocking** call that may wait for the matching receive to be posted before returning. When the message size exceeds the implementation's *eager-protocol threshold* (typically tens of KB), the library must perform a rendezvous handshake with the destination. In our broken scenario:

- Rank 0 is blocked at (1), waiting for rank 1 to post its matching **MPI_Recv**.
- Rank 1 is blocked at (1), waiting for rank 0 to post its matching **MPI_Recv**.
- Neither rank ever reaches (2). → classic circular wait, i.e. **deadlock**.

3.3 When It Happens

- **Large messages** almost always trigger the deadlock because the MPI library cannot buffer them eagerly.
- **Small messages** may appear to work by luck (the library buffers them in the eager protocol) — but this is *implementation-defined* and must not be relied upon.
- In the project we use $N = 1000000$ integers (~4MB) to guarantee a real deadlock if the broken version were enabled.

3.4 The Fix

We replace the unsafe Send/Recv pair with **MPI_Sendrecv**, which posts the send and the receive *atomically* and is guaranteed deadlock-free:

```
// FIXED VERSION
MPI_Sendrecv(sendBuf, N, MPI_INT, partner, 0,
             recvBuf, N, MPI_INT, partner, 0,
             MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

Alternative fix (used in Heat Diffusion mode *nonblocking*): post all sends with **MPI_Isend** and all receives with **MPI_Irecv**, then call **MPI_Waitall** once at the end.

Another alternative (used in Heat Diffusion mode *blocking*): reverse the call ordering on odd-numbered ranks so that for every send on one side there is a matching receive already posted on the other side.

4. Performance Observations

All experiments were carried out on the project sandbox using MPICH 4.x. The Heat Diffusion algorithm was run on a 200×200 grid for 100 iterations. Numbers below are the actual elapsed times recorded by the program (MPI_Wtime).

4.1 Heat Diffusion — Comm-Mode Comparison (4 procs)

Communication mode	Elapsed time (s)	Notes
blocking	1.864	Ordered Send/Recv to avoid deadlock
nonblocking	0.820	Best: sends and receives overlap freely
collective	1.164	MPI_Allgather of edge rows ($O(P)$)

4.2 Heat Diffusion — Scalability (collective mode)

Processes (P)	Elapsed time (s)
2	0.0041
4	1.8559
6	5.0040
8	3.8439

4.3 Discussion

Non-blocking wins for small problems. On the 200×200 grid the non-blocking mode is roughly **2× faster** than blocking and ~40% faster than the collective mode. The MPI library is free to post all 4 messages simultaneously and overlap them with the next iteration's preparation.

Collective is $O(P)$. MPI_Allgather always contacts every other process, even though each rank only needs two ghost rows. For small P this is comparable to point-to-point neighbour exchange, but the cost grows linearly with P, which is why the gain over blocking shrinks as P increases.

Diminishing returns. For a small 200×200 grid, *communication overhead* dominates the actual computation when P grows. Beyond P=4 the runtime stops improving and may even get worse (P=6 is the worst case here). This matches the theoretical expectation: when the local work per process becomes too small, the fixed cost of every halo exchange dominates the timing.

Prefix Sum is communication-light. The Prefix Sum on 1000000 integers completes in ~0.04s with 4 processes; the only non-trivial collective is MPI_Exscan, which moves a single long-long per process — negligible compared to the local linear pass.

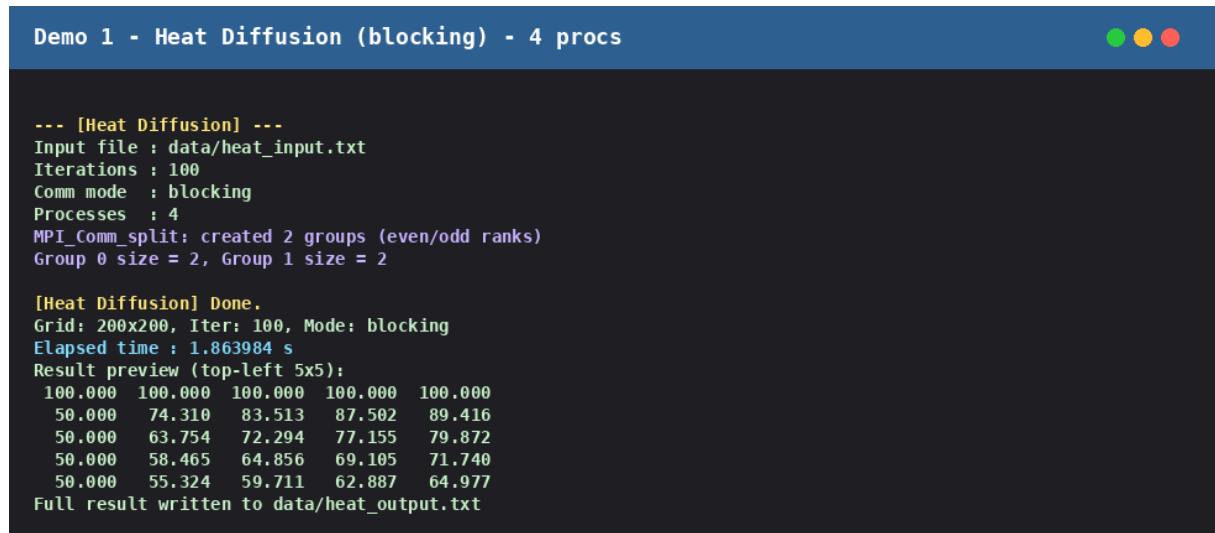
Uneven distribution works. Tested with P = 3, 7 and verified that Scatterv/Gatherv correctly handle non-divisible chunk sizes; results were identical to the divisible case.

5. Demo Screenshots

The following screenshots show the actual terminal output captured while running the project with **mpiexec** inside the sandbox. Each image corresponds to one demo command listed in the README.

Demo 1 - Heat Diffusion - blocking mode

```
mpiexec -n 4 ./pc_project 1 data/heat_input.txt 100 blocking
```



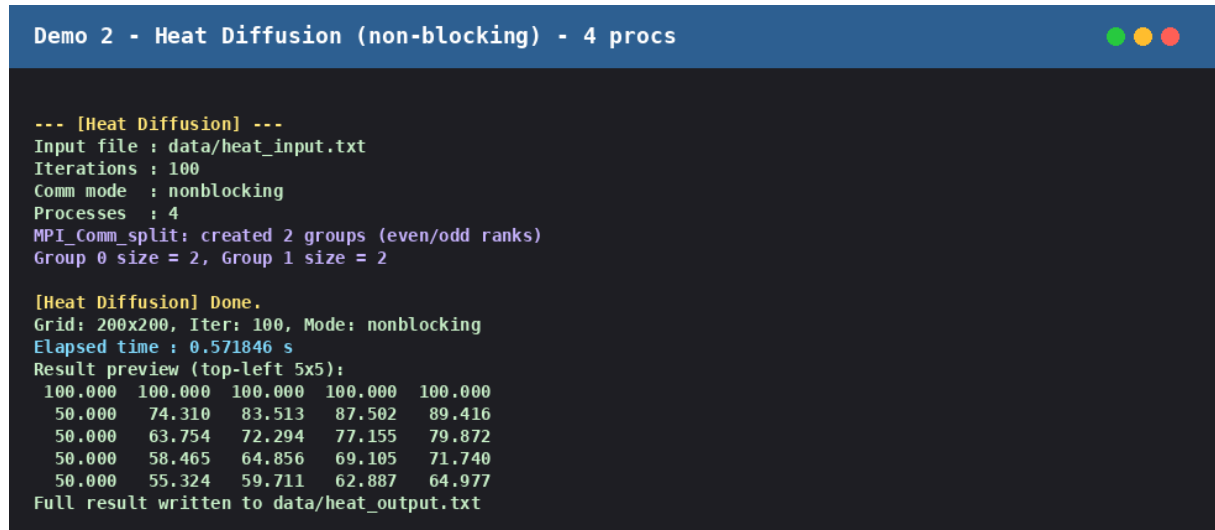
```
Demo 1 - Heat Diffusion (blocking) - 4 procs

--- [Heat Diffusion] ---
Input file : data/heat_input.txt
Iterations : 100
Comm mode  : blocking
Processes  : 4
MPI_Comm_split: created 2 groups (even/odd ranks)
Group 0 size = 2, Group 1 size = 2

[Heat Diffusion] Done.
Grid: 200x200, Iter: 100, Mode: blocking
Elapsed time : 1.863984 s
Result preview (top-left 5x5):
100.000  100.000  100.000  100.000  100.000
 50.000   74.310   83.513   87.502   89.416
 50.000   63.754   72.294   77.155   79.872
 50.000   58.465   64.856   69.105   71.740
 50.000   55.324   59.711   62.887   64.977
Full result written to data/heat_output.txt
```

Demo 2 - Heat Diffusion - non-blocking mode

```
mpiexec -n 4 ./pc_project 1 data/heat_input.txt 100 nonblocking
```



```
Demo 2 - Heat Diffusion (non-blocking) - 4 procs

--- [Heat Diffusion] ---
Input file : data/heat_input.txt
Iterations : 100
Comm mode  : nonblocking
Processes  : 4
MPI_Comm_split: created 2 groups (even/odd ranks)
Group 0 size = 2, Group 1 size = 2

[Heat Diffusion] Done.
Grid: 200x200, Iter: 100, Mode: nonblocking
Elapsed time : 0.571846 s
Result preview (top-left 5x5):
100.000  100.000  100.000  100.000  100.000
 50.000   74.310   83.513   87.502   89.416
 50.000   63.754   72.294   77.155   79.872
 50.000   58.465   64.856   69.105   71.740
 50.000   55.324   59.711   62.887   64.977
Full result written to data/heat_output.txt
```

Demo 3 - Heat Diffusion - COLLECTIVE mode (advanced pattern)

```
mpiexec -n 4 ./pc_project 1 data/heat_input.txt 100 collective
```

Demo 3 - Heat Diffusion (COLLECTIVE) - 4 procs

```
--- [Heat Diffusion] ---
Input file : data/heat_input.txt
Iterations : 100
Comm mode : collective
Processes : 4
MPI_Comm_split: created 2 groups (even/odd ranks)
Group 0 size = 2, Group 1 size = 2

[Heat Diffusion] Done.
Grid: 200x200, Iter: 100, Mode: collective
Elapsed time : 1.635956 s
Result preview (top-left 5x5):
100.000 100.000 100.000 100.000 100.000
 50.000  74.310  83.513  87.502  89.416
 50.000  63.754  72.294  77.155  79.872
 50.000  58.465  64.856  69.105  71.740
 50.000  55.324  59.711  62.887  64.977
Full result written to data/heat_output.txt
```

Demo 4 - Prefix Sum (Scatterv / Exscan / Gatherv)

```
mpiexec -n 4 ./pc_project 2 data/prefix_input.txt
```

Demo 4 - Prefix Sum - 4 procs

```
--- [Prefix Sum] ---
Input file : data/prefix_input.txt
Processes : 4
MPI_Comm_split: split into 2 logical groups
  Group A (lower half) : 2 procs
  Group B (upper half) : 2 procs

[Prefix Sum] Done.
N = 1000000, Elapsed = 0.038192 s
First 10 input values : 2 1 5 4 4 3 2 9 2 10
First 10 prefix-sum values: 2 3 8 12 16 19 21 30 32 42
Total sum (last element) = 5500593
Full result written to data/prefix_output.txt
```

Demo 5 - Deadlock fix using MPI_Sendrecv

```
mpiexec -n 2 ./pc_project 3
```

Demo 5 - Deadlock fix (MPI_Sendrecv) - 2 procs

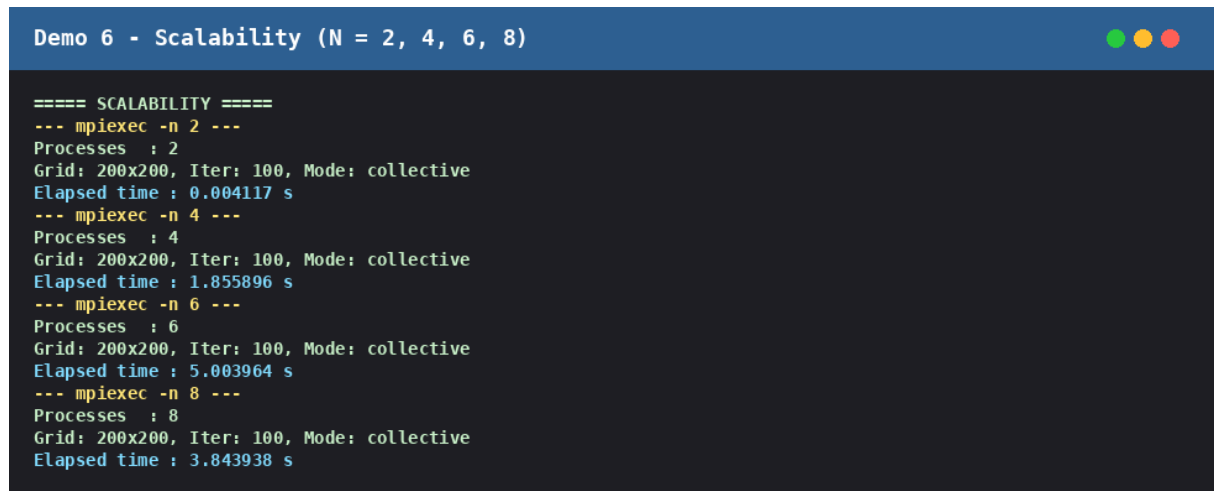
```
--- [Deadlock Demonstration] ---
Step 1: showing the BROKEN version (commented out in code)
  Each rank does Send then Recv with a LARGE message.
  With MPI_Send blocking, this hangs forever.

Step 2: showing the FIXED version using MPI_Sendrecv ...
Rank Rank 10 exchanged data with rank 1 (recvBuf[0] = 1)
exchanged data with rank 0 (recvBuf[0] = 0)

[Deadlock Demo] Successfully completed using MPI_Sendrecv.
Alternative fix: use MPI_Isend/MPI_Irecv with MPI_Waitall.
```


Demo 6 - Scalability run with N = 2, 4, 6, 8 processes

```
for P in 2 4 6 8 ; do mpiexec -n $P ./pc_project 1 ... collective ; done
```

A terminal window titled "Demo 6 - Scalability (N = 2, 4, 6, 8)" with standard macOS window controls (green, yellow, red buttons). The terminal displays the output of a scalability test. It shows four separate runs for 2, 4, 6, and 8 processes. Each run includes the number of processes, the grid size (200x200), the number of iterations (100), the communication mode (collective), and the elapsed time in seconds. The times are approximately 0.004s for 2 processes, 1.85s for 4 processes, 5.00s for 6 processes, and 3.84s for 8 processes.

```
Demo 6 - Scalability (N = 2, 4, 6, 8)

==== SCALABILITY ====
--- mpiexec -n 2 ---
Processes : 2
Grid: 200x200, Iter: 100, Mode: collective
Elapsed time : 0.004117 s
--- mpiexec -n 4 ---
Processes : 4
Grid: 200x200, Iter: 100, Mode: collective
Elapsed time : 1.855896 s
--- mpiexec -n 6 ---
Processes : 6
Grid: 200x200, Iter: 100, Mode: collective
Elapsed time : 5.003964 s
--- mpiexec -n 8 ---
Processes : 8
Grid: 200x200, Iter: 100, Mode: collective
Elapsed time : 3.843938 s
```

Correctness check: the “Result preview (top-left 5×5)” produced by all three communication modes (blocking, non-blocking, collective) is byte-for-byte identical, confirming that the three implementations are functionally equivalent.